

EMAIL SPAM DETECTOR: CHROME EXTENSION WITH PYTHON BACKEND

Anchal Anchal 300364004 – Team Lead

Course

Course Name: 4495 – Applied Research Project

Section: 002

1. Introduction

This report summarizes the modifications and enhancements made to the Spam Detection Chrome Extension project. The goal was to modernize the backend model, upgrade the frontend technology stack, and enable Gmail integration.

2. System Overview

The Spam Detection Chrome Extension with Real-Time Email Scanning is designed to enhance email security by automatically analyzing incoming emails in Gmail to detect spam messages. The system consists of two main components:

Chrome Extension:

Uses **content.js** to monitor Gmail for new emails and extract subject lines and snippets.

Communicates with **background.js**, which sends email data to a backend API for analysis.

Highlights potential spam emails in red within the Gmail inbox.

1. Backend Upgrade: XGBoost Integration

- Previous Approach: Basic classifier (likely Naive Bayes or similar).
- New Implementation: Migrated the spam classification logic to use the XGBoost algorithm.
- Steps Taken:
 - Installed xgboost and joblib.
 - Trained an XGBoost classifier using spam_cleaned.csv.
 - Saved the trained model and TF-IDF vectorizer for reuse.
 - Updated app.py Flask API to load and predict using the saved model.

2. Frontend Replacement: AngularJS UI

- Previous Setup: Presumably static HTML.
- New Setup: Basic AngularJS app created for spam detection interface.
- Files Created:
 - index.html – Basic interface with textarea and button.

- app.js – Defines the AngularJS module.
- controller.js – Manages HTTP requests to the backend API.

3. Gmail API Integration

- Objective: Fetch Gmail messages and evaluate spam status using the backend API.
- Tool Used: Gmail API with OAuth2 via google-auth-oauthlib and google-api-python-client.
- New Script: gmail_fetcher.py
 - Authenticates the user.
 - Fetches recent messages from Gmail.
 - Sends email snippets to Flask API.
 - Displays the spam classification result in the console.

Technology Stack:

- Frontend (Chrome Extension): JavaScript (content & background scripts), HTML, CSS
- Backend (Spam Detection API): Python, Flask, scikit-learn, joblib
- Machine Learning Model: Trained on spam datasets using TfidfVectorizer + Naïve Bayes
- Deployment & Testing: GitHub, ngrok (for public API testing)

The system provides a seamless real-time solution for detecting spam in Gmail while maintaining a lightweight and user-friendly interface.

Automatic Spam Detection System

- Initial Design: Manual execution of the Gmail script was required.
- Upgrade Implemented: A background polling loop was added to automatically check Gmail for new emails at regular intervals (e.g., every 5 minutes).
- Approach Used:
 - A scheduler using time.sleep() was embedded in the script.
 - The function fetch_and_check() now runs in a loop, simulating real-time detection.
- Alternative Considered:

- Gmail's Push Notification using Google Pub/Sub for real-time alerts was researched.
- Due to its complexity (requires HTTPS endpoint, billing, and domain verification), this was not implemented but remains a future enhancement.

Work Date/Hours Logs

Date	Hours	Descriptions
Jan 14 2025	2.4	Started searching for the project topic
Jan 15 2025	2	Looking for suitable topic
Jan 17 2025	3	Collected Information and made document about the fundamentals
Jan 20 2025	4	Got some ideas from professor and started looking forward on project
Jan 22 2025	4	Search for the tools and language to use
Jan 23 2025	3.5	Started working on the project, created json files for plugin
Jan 25 2025	3.5	created html and css pages for the chrome extension and some changes in chrome plugin
Jan 27 2025	4	studied some topics from machine laeaning and tried to implement in the project
Jan 29 2025	3	Faced diccifulty to push files to github and imported libraries
Jan 31 2025	3.5	Learned Machine laeaning Bay's theorem and did some practice to implement in the project
Feb 02 2025	4	Learned how to create chrome extension and tried to create it
Feb 03 2025	3	Created chrome extension files and uploaded to chrome extension
Feb 05 2025	2.5	Found out errors encountered while data cleaning
Feb 6 2025	3	Pushed files to github
Feb 9 2025	1	Created Report and checked the files done
Feb 12,2025	2	Installed libraries like blinker click and colorama
Feb 16,2025	2	imported metadata worked on api
Feb 17,2025	3	Set ted Up the Backend (Flask API)
Feb 23,2025	3	Pushed files on github and made video to upload.

Feb 25,2025	4	Studied Chrome extension architecture and reviewed Gmail content script integration.
Feb 28,2025	3.5	Created Manifest.json, added permissions for Gmail, and set up basic Chrome extension structure.
March 2,2025	3	Started development of content.js for scanning Gmail inbox. Implemented email extraction from Gmail UI and tested script injection.
March 4,2025	4	Worked on background.js to handle email content processing and API communication.
March 6,2025	3.75	Developed Flask API to receive email data for spam classification.
March 7,2025	4	Saved trained model as spam_model.pkl and integrated with Flask API again.
March 9,2025	4.2	Debugged API integration, identified issues with joblib vs pickle loading. Implemented MutationObserver in content.js to detect live email updates.
March 10 ,2025	3	Conducted initial testing, discovered API response lag issues. Optimized API response time by refining text preprocessing steps.
March 12 2025	3	Implemented email highlighting feature for detected spam messages. Deployed Flask API temporarily using ngrok for public access testing.
March 13 ,2025	4	Debugged scikit-learn version conflicts, considered model retraining. Refined spam classification pipeline, improved accuracy metrics.
March 15,2025	3.75	Integrated all extension components and tested Gmail real-time scanning. Conducted debugging and performance tuning for API interactions.
March 16,2025	5	Pushed files to git and made report. Tried to resolve errors and run API.
Mar 19, 2025	3	Researched and compared alternative browser automation tools (e.g., Puppeteer vs.

		Playwright) for potential integration into the spam detection pipeline.
Mar 20, 2025	3	Designed a fallback mechanism for email extraction when DevTools-based methods fail. Began prototyping a simplified heuristic parser for HTML email content.
Mar 21, 2025	2.5	Started implementing rate-limiting logic for API requests to prevent browser lag and avoid spam filter triggers during testing.
Mar 22, 2025	3	Developed and tested a feature to tag spam confidence scores on the frontend with visual indicators (e.g., color-coded flags).
Mar 23, 2025	4	Refactored project structure for better maintainability. Modularized email extraction logic and restructured model loading pipeline to support future updates. Also made report 3.
Mar 24, 2025	4	Reviewed initial project structure, identified tech stack changes needed.
Mar 25, 2025	4.5	Replaced ML model with XGBoost, wrote training and saving logic.
Mar 26, 2025	3.5	Updated Flask backend (app.py) to use XGBoost and serve predictions.
Mar 27, 2025	2	Built AngularJS frontend files and tested integration with backend API.
Mar 28, 2025	5	Researched Gmail API, set up credentials, and began script for email fetching.
Mar 29, 2025	4.5	Tried and tested gmail_fetcher.py script.
Mar 30, 2025	4	Compiled report and documented worklog. Final testing and packaging.

Current Challenges and Next Steps

- Deploy the Flask backend to a live server (e.g., Heroku, AWS).
- Package AngularJS app into a Chrome Extension popup.
- Secure API endpoints with authentication.
- Upgrade to Gmail Pub/Sub notifications for real-time spam detection.
- Auto-label spam emails in Gmail using Gmail API.